End-to-End Architectural Blueprint and Implementation Strategy: Tasker iOS Weekly Operating Layer

The transition of a sophisticated productivity application from a purely execution-focused task manager to an intentional, outcome-driven planning system requires a profound architectural alignment. The initiative to introduce a Weekly Planning and Reflection subsystem into the Tasker iOS application is an exercise in additive software engineering, requiring strict adherence to the existing repository structures, dependency injection graphs, and data mutation contracts.[1] This comprehensive report outlines the end-to-end implementation blueprint, mapping the product specifications directly to the codebase encapsulated within the To Do List/ workspace.[2] The foundational directive is absolute: do not architect a parallel, disparate planner application; rather, extend the core semantic models, Core Data schemas, use cases, and view models to seamlessly integrate the loop of capturing, deciding, starting, resuming, and reflecting.[1]

Architectural Stance and Repository Grounding

The Tasker iOS repository is meticulously structured to enforce a clean separation of concerns, heavily utilizing modular design patterns defined within its architectural documentation, specifically docs/architecture/state-repositories-and-services-v2.md.[2] The application logic is segmented into Domain, UseCases, Presentation, Views, and Persistence layers.[1] An execution-first paradigm defines the current application state, prioritizing instantaneous task capture, rigorous dependency and tag graph management, and a canonical TaskDefinition-centric write model.[2]

Introducing a weekly operating layer must respect this established equilibrium. The product thesis asserts that weekly structures must reside in the background, surfacing necessary context without polluting the high-velocity daily execution layer.[1] Creating new, duplicate abstractions to handle tasks assigned to a "week" would violate the existing repository seams. Instead, the implementation strategy mandates utilizing the current ViewModels—such as HomeViewModel, TaskDetailViewModel, and ProjectManagementViewModel—as the primary integration interfaces.[1] Read-heavy state distributions, particularly for the gamification and analytics systems, must derive their inputs directly from the existing analytics engines, capitalizing on signals like momentum, due pressure, and completion mix already present in the system.[2]

The Philosophy of Additive Slices

The weekly system is designed as an additive slice rather than a monolithic rewrite. This means that instead of restructuring the temporal logic of the entire application, the engineering approach extends existing concepts where the weekly feature is a true property of an entity,

while strictly isolating week-specific artifacts into new, first-class entities.[1] This preserves the integrity of the daily execution layer—comprising the Today view, focus strips, and due agendas—ensuring that tactical, execution-oriented interfaces are not overwhelmed by strategic, planning-oriented metadata.[1] The architectural boundary condition is firmly established: the Home view must not be transmuted into a dense weekly dashboard; it must remain a tactical surface that utilizes compact entry points to bridge the user into dedicated, isolated planning and review flows.[1]

Concise Repo-Mapped Implementation Plan

Before executing the phased implementation, it is critical to map the required domain additions directly to the existing repository structure. This precise file-system mapping ensures minimal architectural disruption and guarantees that all new abstractions align with the existing dependency injection containers.

Layer	Existing File to Modify	New File to Create	Architectural Responsibility
Domain / Models	To Do List/Domain/Models/Task.swift [2]	TaskPlanningBucket.swift	Defines the temporal horizon enumeration.[1]
Domain / Models	To Do List/Domain/Models/Project.swift [2]	WeeklyPlan.swift, WeeklyOutcome.swift, WeeklyReview.swift, ReflectionNote.swift, WeeklyMomentumSummary.swift	Establishes the new first-class domain entities for the weekly lifecycle.[1]
Domain / Interfaces	TaskReadModelRepositoryProtocol.swift [2]	WeeklyPlanRepositoryProtocol.swift, WeeklyOutcomeRepositoryProtocol.swift, WeeklyReviewRepositoryProtocol.swift, ReflectionNoteRepositoryProtocol.swift	Defines the contracts for data persistence and retrieval boundaries.[1]

Data / Persistence	To Do List/Data/Persistence/CoreDataStack.swift [3]*	*Core Data Model Version (V3)*	Implements the physical SQLite schema mappings and migration paths.[1]
UseCases / Weekly	*N/A (New logical grouping)*	GetActiveWeeklyPlanUseCase.swift, CreateOrUpdateWeeklyPlanUseCase.swift, BuildWeeklyPlannerSuggestionsUseCase.swift, MoveTaskPlanningBucketUseCase.swift, BulkMoveTaskPlanningBucketUseCase.swift	Encapsulates the transactional business logic for orchestrating week creation and task staging.[1]
UseCases / Reflection	*N/A (New logical grouping)*	CreateReflectionNoteUseCase.swift, GetReflectionNotesUseCase.swift	Isolates the prompt-driven journaling logic.[1]
UseCases / Momentum	*N/A (New logical grouping)*	CalculateWeeklyMomentumUseCase.swift, EstimateWeeklyCapacityUseCase.swift	Houses the derived algorithmic evaluations for system health and overload detection.[1]
Presentation / ViewModels	HomeViewModel.swift [2], InsightsViewModel.swift [2]	WeeklyPlannerViewModel.swift, WeeklyReviewViewModel.swift, ReflectionNoteComposerViewModel.swift, WeeklyHistoryDetailViewModel.swift	Manages UI state, handles in-memory drafting for planner interfaces, and formats data for SwiftUI consumption.[1]

Views / Weekly	*Existing UI structural directories*	WeeklyPlannerView.swift, WeeklyReviewView.swift, WeeklyOutcomeEditorView.swift, WeeklyCapacityMeterView.swift, WeeklyMomentumCardView.swift, ReflectionNoteComposerSheet.swift	Renders the visual hierarchy and interprets user gestures.[1]

(Note: Assumed paths like CoreDataStack are extrapolated from standard architecture within the repository structure identified).

Domain Model Expansion and Extensibility

The foundational modifications occur within the Swift domain models. The application's core entity, the TaskDefinition, relies on a strict write-model architecture that manages canonical identity and mutation sequences.[2] Altering this model requires absolute precision to avoid cascading failures in existing read queries.

Modifying the Execution Primitives

The TaskDefinition model must absorb properties that dictate its behavior within a weekly cycle. The addition of the planningBucket property is paramount. By defining this as a TaskPlanningBucket enumeration—encompassing today, thisWeek, nextWeek, later, and someday—the application replaces brittle, string-based tagging architectures with a highly typed, queryable temporal property.[1] This natively integrates the planning horizon into the task's identity.

Furthermore, a weeklyOutcomeID of type UUID? is introduced to forge a direct, relational linkage between a granular execution action and a strategic weekly sub-goal.[1] To power the advanced momentum and capacity algorithms, the system must track task deferrals. The deferredFromWeekStart (Date?) and deferredCount (Int) fields are appended to the task model.[1] These properties serve as the raw signals for identifying backlog anxiety, allowing the analytics engine to quantify unresolved commitments over time.[1]

Concurrently, the Project.swift model is expanded to incorporate deep motivation anchors.[1] The addition of motivationWhy, motivationSuccessLooksLike, and motivationCostOfNeglect as optional strings transforms the project entity from a mere grouping mechanism into an intentional strategic container.[1] This ensures that the "why" of the work is preserved and travels natively with the project data graph, preventing the cognitive fade that occurs when users are

exposed only to isolated task views.[1]

Introducing Weekly Lifecycle Entities

To prevent the existing models from collapsing under the weight of lifecycle management, distinct, first-class domain models are authored for the weekly system.

Domain Model	Architectural Purpose	Memory and Concurrency Considerations
WeeklyPlan	Represents the mutable shell of the active operating week. Contains bounds, a focus statement, and selected habit identifiers.[1]	Evaluated purely in memory during planning states. Persisted only upon explicit confirmation to avoid rapid UI-driven thrashing.
WeeklyOutcome	Defines structural, user-defined sub-goals. Restricted to a maximum of three entries to enforce focus.[1]	Handled as ordered arrays within the parent plan. Requires strict index recalculations during drag-and-drop UI interactions to prevent sorting collisions.
WeeklyReview	Functions as an immutable, historical artifact capturing the end-of-week state, including wins, blockers, and perceived ratings.[1]	Transitioned from a mutable draft state to an immutable completed state upon the execution of the weekly closure use case.
ReflectionNote	A lightweight, flexible entity designed for context capture, utilizing polymorphic linkages (linkedTaskID, linkedProjectID).[1]	Engineered for low-latency writes. These entities must be decoupled from the primary task save pipelines to ensure journaling never blocks UI threads.[1]
WeeklyMomentumSummary	A strictly derived structure providing algorithmic health metrics, blending quantitative task data with	Not mapped to Core Data initially. Computed on the fly by traversing recent review and completion history to

	narrative momentum framing.[1]	prevent data state divergence.[1]

Core Data Schema Evolution and Persistence Strategies

Translating these rich Swift domain structures into the SQLite-backed Core Data implementation necessitates careful orchestration to ensure backward compatibility and minimal read latency. The architectural strategy mandates deploying a new version within the existing V3 model family, relying heavily on additive, lightweight migration techniques.[1]

Migration Mechanisms and Entity Definitions

The schema migration must introduce the new attributes to the Task and Project entities without triggering heavy processing or blocking the application launch cycle.

1. **Task Entity Migration:** The planningBucketRaw string attribute is appended to the Task entity, providing the persistent backing for the Swift domain enumeration. A critical aspect of the migration is defaulting logic: to prevent sudden task disappearance or backlog confusion, all existing open tasks without an assigned bucket must be dynamically defaulted to thisWeek upon the initial schema transformation.[1] Optional fields such as weeklyOutcomeID and integer attributes like deferredCount are appended with safe default initializers (null and zero, respectively).[1]
2. **Project Entity Migration:** The motivational text anchors (motivationWhy, etc.) are added as optional strings. To maintain database efficiency, a stringent sanitization layer must be implemented within the repository's save method, ensuring these strings are aggressively trimmed of trailing whitespace and nullified if left completely empty, preventing the storage of empty string objects.[1]
3. **New Relational Tables:** The creation of WeeklyPlanEntity, WeeklyOutcomeEntity, WeeklyReviewEntity, and ReflectionNoteEntity establishes the structural backend.[1] These entities require highly optimized indices. The WeeklyPlanEntity necessitates an index on the weekStartDate to rapidly retrieve the current active plan.[1] WeeklyOutcomeEntity relies on a cascading deletion rule originating from its parent WeeklyPlanEntity to prevent orphaned rows in the SQLite store, while utilizing a orderIndex column to maintain deterministic retrieval sorting.[1]

By adhering strictly to the existing Core Data framework and avoiding parallel persistence solutions (such as sidecar JSON files or separate databases), the implementation leverages existing memory faulting capabilities and background context synchronization inherent in the repository architecture.[1]

Repository Contract Extensions and Mutational

Boundaries

The interface boundaries, defined within the Domain/Interfaces directory, serve as the explicit contracts between the pure business logic and the database implementation.[2] The foundational principle dictating the repository modifications is the absolute avoidance of one-off, localized mutation methods.[1]

Centralizing Task Mutations

The TaskDefinitionRepository retains its status as the exclusive mutational pipeline.[1] Rather than polluting the protocol with redundant functions such as updateTaskPlanningBucket(to:) or linkTaskToOutcome(), these attributes are incorporated seamlessly into the standard task update payload. This design ensures that all modifications pass through the existing transactional command sequence—the strict propose/confirm/apply/undo semantics documented within the Tasker architecture.[2] Consequently, updating a task's planning bucket triggers the identical validation logic, timestamp updating, and cache invalidation routines as modifying its title or tags, guaranteeing state consistency across the application.

Conversely, the read paths governed by the TaskReadModelRepositoryProtocol necessitate targeted extensions.[2] To efficiently populate the planner and review surfaces, the read query interfaces must accept advanced predicate filters for planningBuckets and weeklyOutcomeID.[1] This prevents the system from loading the entire repository into application memory to execute local Swift array filtering, thereby preserving performance on devices with substantial task backlogs.

Architecting New Repositories

The first-class weekly entities demand dedicated repository protocols, fully integrated into the existing dependency injection container.[1]

Repository Protocol	Contracted Responsibilities	Systemic Implications
WeeklyPlanRepositoryProtocol	fetchActivePlan(), fetchPlan(byDate:), savePlan(), deletePlan()	Operates as the backing store for the week's foundational shell. Must enforce singleton-like logic for any given week boundary to prevent conflicting plan artifacts.[1]
WeeklyOutcomeRepository	fetchOutcomes(forPlanID:),	Employs atomic

Protocol	saveBatch(), deleteOutcome()	batch-saving mechanisms to accommodate complex UI reordering operations, preventing UI/database desynchronization.[1]
WeeklyReviewRepositoryProtocol	fetchReview(forPlanID:), saveDraft(), completeReview()	Manages explicit state transitions. The saveDraft() method supports asynchronous, multi-session reviews without prematurely closing the week or firing XP triggers.[1]
ReflectionNoteRepositoryProtocol	queryNotes(byLinkID:, type:), saveNote(), deleteNote()	Optimized for high-throughput, asynchronous writes across background Core Data contexts, ensuring rapid-fire UI journaling flows are never blocked.[1]

Orchestrating Business Logic via Domain Use Cases

The clean architecture of the Tasker application relies profoundly on Use Cases to decouple complex business rules from both the interface layers and the database implementations.[1] The introduction of the UseCases/Weekly, UseCases/Reflection, and UseCases/Momentum namespaces localizes the complex state transformations required for the weekly lifecycle.[1]

Staging and Horizon Management Logic

The initialization of the weekly planning interface is driven by GetActiveWeeklyPlanUseCase and CreateOrUpdateWeeklyPlanUseCase.[1] When a user invokes the planner, BuildWeeklyPlannerSuggestionsUseCase executes highly complex background logic, parsing historical behavior and integrating signals from the HomeAIActionCoordinator to pre-populate the staging environment with suggested focus items and high-priority backlog tasks.[1]

The movement of tasks across temporal bounds is governed by MoveTaskPlanningBucketUseCase and its critical counterpart, BulkMoveTaskPlanningBucketUseCase.[1] The bulk use case is technically vital; during a weekly reset or an aggressive deferral operation where dozens of tasks are shifted to later, executing individual save commands would oversaturate the Core Data thread and severely impact UI

responsiveness. The bulk use case orchestrates these transitions via optimized batch updates.[1]

The End-of-Week Transaction

The closure of the weekly cycle is the most transactionally complex sequence in the application. BuildWeeklyReviewSummaryUseCase performs extensive cross-repository reads, analyzing the completed execution mix, the continuity of habits, and the persistence of unfinished tasks to assemble a comprehensive pre-filled draft.[1] This satisfies the rigorous UX requirement that the review must be completely actionable with zero mandated text input.[1]

Upon user confirmation, CompleteWeeklyReviewUseCase executes the final transactional commit. It finalizes the WeeklyReview entity, transitions the WeeklyPlan to a closed state, resolves all staged deferral operations, and crucially, broadcasts the necessary global completion events to trigger gamification updates.[1] Subsequently, GenerateNextWeekFromReview seamlessly instantiates the upcoming planning shell, ensuring the user is never left in a structurally void state.[1]

Algorithmic Momentum and Overload Diagnostics

The mathematical and narrative core of the system resides in the CalculateWeeklyMomentumUseCase.[1] Momentum within this architecture transcends simple volumetric task completion. The algorithm evaluates a synthesis of active outcome progress, behavioral recovery patterns, habit consistency, and the psychological weight of carry-over pressure.[1] This ensures that users who experience a catastrophic week but engage in healthy recovery behavior—such as systematically deferring tasks and resetting expectations—are algorithmically rewarded with a positive momentum narrative rather than penalized.[1]

Concurrently, the EstimateWeeklyCapacityUseCase functions as a proactive diagnostic tool. By analyzing historical completion velocities against the current volume of tasks staged within the thisWeek bucket, the use case calculates a targetCapacity threshold.[1] If the threshold is breached, the use case emits a localized state change that illuminates an "overload chip" in the UI, preempting burnout before execution begins.[1]

Presentation Layer: ViewModel Isolation and State Management

Integrating these complex operations into the user interface demands precise control over the SwiftUI view models. The primary architectural risk is overwhelming the core execution views with planning logic.

Defending the Home Execution Environment

The HomeViewModel serves as the central hub of daily operation.[2] To preserve its speed, the HomeViewModel is strictly forbidden from manually computing planner states, aggregating outcomes, or managing horizon sorting logic.[1] Instead, it is injected with a lightweight summary

listener that passively consumes pre-calculated state packets, rendering a minimal, unobtrusive weekly status card.[1] This card displays dynamic calls to action based on the lifecycle stage (e.g., "Plan week" vs. "Review week") and houses the capacity overload chip, ensuring the Home screen remains overwhelmingly focused on the "Now" rather than the distant strategic horizon.[1]

Similarly, the AddTaskViewModel must retain its instantaneous quick-capture capabilities. Planning complexity is suppressed behind progressive disclosure mechanisms. Only when a user intentionally expands the creation interface does the view model expose bucket and outcome selection.[1] A critical piece of reactive logic is implemented here: if a user selects a weeklyOutcomeID, the AddTaskViewModel utilizes Combine pipelines or didSet observers to automatically default the planningBucket to thisWeek, removing redundant keystrokes.[1]

The TaskDetailViewModel, which inherently handles rich entity metadata and autosave debouncing, naturally absorbs the inline bucket selections and outcome linkages.[1] Meanwhile, the ProjectManagementViewModel is extended to bind the motivationWhy, motivationSuccessLooksLike, and motivationCostOfNeglect string properties to its internal state, seamlessly persisting them through the established project save pipelines.[1]

Architecting Dedicated Lifecycle ViewModels

The profound difference between high-speed execution and thoughtful planning dictates the creation of deeply isolated view models for the weekly operations.

Dedicated ViewModel	State Management Mechanics	Architectural Imperatives
WeeklyPlannerViewModel	Orchestrates local memory staging. Fetches candidates, outcomes, and habits upon initialization.[1]	Must never bind real-time structural edits to the database. All user actions (dragging tasks between buckets, drafting outcomes) modify temporary local state. The data is only mapped back to the domain and committed to Core Data upon explicit activation of the save footer.[1]
WeeklyReviewViewModel	Manages the sequential, paginated or scrolling states of the review summary, draft	Implements robust crash-recovery through automated draft saving.

	wins/blockers, and carry-forward decisions.[1]	Handles massive state aggregations but relies on the pre-calculation capabilities of the injected use cases to maintain 60FPS UI scrolling performance.[1]
InsightsViewModel	Operates as a strictly read-only historical consumer. Formats the WeeklyMomentumSummary data for graphical rendering.[1]	Displays the week intensity strip, recovery insights card, and carry-over pressure diagnostics. Explicitly devoid of mutational operations to preserve architectural separation.[1]

The ReflectionNoteComposerViewModel represents an entirely independent, transient state manager. Designed to populate a dismissible modal sheet, it manages the rapid selection of prompt chips and textual input, immediately dispatching save commands to the background ReflectionNoteRepositoryProtocol upon completion or deliberate skipping, ensuring no interruption to concurrent task completion flows.[1]

Deep Integration: Gamification, Intelligence, and System Events

The weekly architecture must interface seamlessly with the proprietary Tasker engagement systems, specifically the Experience Points (XP) engine and the Eva AI coordinator.[1]

XP Engine Mechanics and Anti-Abuse Logic

The existing gamification system triggers on events such as daily completions and retrospective analysis.[2] This system is extended to intercept new analytical signals originating from the weekly use cases.[1] New discrete XP events are mapped: weeklyPlanCreated, weeklyReviewCompleted, reflectionNoteCaptured, meaningfulCarryForwardCleanup, and weeklyResetCompleted.[1]

A critical engineering mandate is the implementation of anti-abuse constraints. The specification strictly prohibits awarding XP for arbitrary user interface interactions, such as repeatedly shuffling the same task back and forth between the thisWeek and nextWeek buckets.[1] Consequently, the event dispatcher within the view models must employ strict debouncing, or more effectively, only broadcast XP-eligible events upon the definitive closure of a transactional cycle (e.g., when the CompleteWeeklyReviewUseCase resolves successfully).[1]

Intelligence Coordination via HomeAIActionCoordinator

The HomeAIActionCoordinator functions as the integration point for Eva, the application's assistant layer.[1] Eva's weekly involvement is engineered as a structured proposal system rather than an autonomous mutating agent.

1. **Ask Mode:** Functions as a pure read context. The coordinator queries historical reviews and active plan states to construct natural language responses to user inquiries regarding their workload.[1]
2. **Suggest Mode:** This constitutes the highest technical complexity. The coordinator utilizes BuildWeeklyPlannerSuggestionsUseCase to generate a secondary, virtual plan state in memory. The UI subsequently renders a visual "diff," clearly distinguishing between the user's current staged plan and Eva's proposed re-prioritizations or capacity-based deferrals.[1]
3. **Apply Mode:** To guarantee user autonomy, the application of Eva's suggestions must pass through a strict gating mechanism, requiring explicit user confirmation before the coordinator issues the final commit commands to the repository layer.[1]

Phased Execution Plan: File-Level Implementation Strategy

Deploying such a comprehensive layer within a live, production repository requires a tightly orchestrated, phased execution plan.[1] Attempting a monolithic release drastically increases the probability of regressions within the critical execution pathways. The plan is executed across six isolated sprints, moving from data foundation outward to intelligence and UI.

Phase 1: Foundational Domain and Data Access (Sprint 1)

Objective: Establish the underlying persistence and domain model extensions with absolute zero user interface visibility, ensuring legacy compatibility.[1]

- **Touched Files:**
  - To Do List/Domain/Models/Task.swift: Add planningBucket, weeklyOutcomeID, deferredFromWeekStart, deferredCount.
  - To Do List/Domain/Models/Project.swift: Add motivation strings.
  - To Do List/Data/Persistence/CoreDataStack.swift: Register V3 Model schema.
  - *New Files:* TaskPlanningBucket.swift, entity models, and all repository interfaces (WeeklyPlanRepositoryProtocol, etc.).
- **Migration Details:** Execute the V3 lightweight Core Data migration. Implement the crucial background loop that scans the database and defaults all existing, unresolved task models to thisWeek to ensure continuity.[1]
- **Tests:** Implement robust unit tests verifying that JSON decoding of legacy TaskDefinition payloads correctly falls back to the default bucket state. Write in-memory Core Data context tests asserting that the optional motivation string additions do not cause

null-pointer exceptions during batch save operations.[1]

- **Assumptions:** The V3 migration path must not execute on the main thread during the UIApplication launch sequence to prevent watchdog terminations on devices with massive legacy databases.

Phase 2: Core Weekly Planner Implementation (Sprint 2)

Objective: Bring the planning loop online, enabling memory staging and database saving without disrupting daily flows.[1]

- **Touched Files:**
  - UseCases/Weekly/GetActiveWeeklyPlanUseCase.swift
  - UseCases/Weekly/CreateOrUpdateWeeklyPlanUseCase.swift
  - Presentation/ViewModels/Home/HomeViewModel.swift: Add the compact status card listener.
  - Presentation/ViewModels/Weekly/WeeklyPlannerViewModel.swift
  - Views/Weekly/WeeklyPlannerView.swift
- **Migration Details:** N/A (Data layer stabilized in Phase 1).
- **Tests:** Develop ViewModel tests specifically targeting the WeeklyPlannerViewModel. Assert that modifying the internal bucket arrays via UI drag-and-drop actions does *not* invoke the TaskDefinitionRepository until the explicit save() command is triggered.[1] Validate the HomeViewModel injection sequence to ensure the card updates instantly post-save.[1]
- **Assumptions:** Memory staging is feasible for workloads up to 500 tasks per active week before performance degradation necessitates pagination within the planner UI itself.

Phase 3: Closure and Carry-Forward Execution (Sprint 3)

Objective: Implement the historical resolution of the week, enabling review summaries and complex state resetting.[1]

- **Touched Files:**
  - UseCases/Weekly/BuildWeeklyReviewSummaryUseCase.swift
  - UseCases/Weekly/CompleteWeeklyReviewUseCase.swift
  - Presentation/ViewModels/Weekly/WeeklyReviewViewModel.swift
  - Views/Weekly/WeeklyReviewView.swift
  - To Do List/Domain/Services/GamificationService.swift (or relevant XP dispatch center): Register the new lifecycle event triggers.[1]
- **Migration Details:** Introduce the WeeklyReview historical entities to the database structure.
- **Tests:** XCUITest automation traversing the full lifecycle: Plan Creation -> Task Deferral -> Review Initiation -> Review Completion. Ensure that deferred tasks correctly increment their deferredCount integer and correctly adjust their deferredFromWeekStart dates within the database after the CompleteWeeklyReviewUseCase resolves.[1]

- **Assumptions:** The review aggregation logic can perform multi-table joins (Tasks, Projects, Habits, History) rapidly enough to render the pre-filled review screen within one second of invocation.

Phase 4: Purpose Integration and Asynchronous Memory Capture (Sprint 4)

Objective: Deploy the deep motivation anchors and qualitative reflection logic, enhancing diagnostic capabilities.[1]

- **Touched Files:**
  - Presentation/ViewModels/ProjectManagementViewModel.swift
  - Presentation/ViewModels/ReflectionNoteComposerViewModel.swift
  - UseCases/Reflection/CreateReflectionNoteUseCase.swift
  - Views/Weekly/ReflectionNoteComposerSheet.swift
  - Views/Project/ProjectDetailView.swift
- **Migration Details:** Enable the polymorphic linkedID fields within the reflection table.
- **Tests:** Execute UI tests validating that triggering a reflection note modal immediately post-task-completion does not block or conflict with the underlying task's undo timer or auto-dismissal logic.[1]
- **Assumptions:** Reflection notes are transient enough that they do not require complex edit-history versions; they operate on a simple create/overwrite paradigm.

Phase 5: Diagnostic Insights and Capacity Modeling (Sprint 5)

Objective: Transform raw execution data into derived, algorithmic narrative signals.[1]

- **Touched Files:**
  - UseCases/Momentum/CalculateWeeklyMomentumUseCase.swift
  - UseCases/Momentum/EstimateWeeklyCapacityUseCase.swift
  - Presentation/ViewModels/InsightsViewModel.swift
  - Views/Weekly/WeeklyCapacityMeterView.swift
- **Migration Details:** Ensure derived summaries query efficiently against historical indices.
- **Tests:** Implement extensive boundary tests against the momentum mathematical models. Supply mocked datasets showcasing 0% completion but high intentional carry-forward rates, and assert that the narrative engine outputs a "recovery" focused narrative rather than a penalty score.[1]
- **Assumptions:** Capacity algorithms initially rely on simple rolling-average completion velocities over the prior four weeks, rather than complex machine-learning derivations.

Phase 6: Intelligence Maturation and QA Hardening (Sprint 6)

Objective: Finalize assistant integration, optimize performance thresholds, and solidify the system architecture for scale.[1]

- **Touched Files:**
  - To Do List/Presentation/Coordinators/HomeAIActionCoordinator.swift
  - UseCases/Weekly/BuildWeeklyPlannerSuggestionsUseCase.swift
  - *(Various QA tuning and performance optimizations across all files).*
- **Migration Details:** Final integrity checks across extensive sandbox environments.
- **Tests:** Conduct load testing utilizing simulated multi-year task histories (10,000+ objects).[2] Profile memory consumption during the execution of BuildWeeklyPlannerSuggestionsUseCase to guarantee that memory spikes remain under system kill thresholds. Validate the Eva suggestion "diff" visually correctly maps additions and removals.[1]
- **Assumptions:** The underlying intelligence coordinator already possesses robust text-to-intent parsing capabilities, enabling it to invoke the suggestion use case reliably.

Architectural Validation and Automated QA Strategy

Ensuring the stability of a core execution application demands a multi-tiered validation approach.[1] The existing xcodebuild test execution framework (-workspace Tasker.xcworkspace -scheme "To Do List" -destination 'platform=iOS Simulator,name=iPhone 16' -only-testing:TaskerTests) must be expanded to encompass the new boundaries.[2]

Unit testing forms the initial defensive line. It strictly isolates the Swift domain structures, validating that the Enum parsing for planning buckets correctly resolves defaults.[1] Critical business rules, such as the system forcefully rejecting a fourth weekly outcome or correctly interpreting capacity limits, must be proven within milliseconds using mocked datasets without ever touching Core Data.

Integration testing targets the repository mappings. These tests instantiate in-memory SQLite stores to verify the V3 migration path.[1] By synthetically injecting thousands of entities, the tests assert that the TaskReadModelRepositoryProtocol successfully filters via the new array-based planningBuckets predicates without triggering full table scans, guaranteeing consistent performance metrics across varied device capabilities.

Finally, End-to-End User Interface validation (XCUITest) guarantees the structural integrity of the complex UX workflows. These automated tests simulate the exact user gestures required to instantiate a weekly plan, stage tasks between horizontal boundaries, and fully complete the review cycle complete with carry-forward instructions.[1] The critical assertion within these tests ensures that executing deep planning logic via the dedicated view models generates the expected momentum narrative inside the InsightsViewModel without inadvertently altering or corrupting the HomeViewModel daily execution queue.[1]

Works cited

1. tasker_weekly_planning_reflection_combined_spec.pdf
2. Saransh-Sharma/Tasker: Gamified Tasks & Productivity ... - GitHub, accessed

April 11, 2026, https://github.com/Saransh-Sharma/Tasker
3. accessed January 1, 1970, https://raw.githubusercontent.com/Saransh-Sharma/Tasker/main/To%20Do%20List/Data/Persistence/CoreDataStack.swift